Intermediate (watch the model choose between two tools) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

`bind_tools([tool_a, tool_b])` hands the model both descriptions at once, and tool selection happens entirely inside the model — your code has no `if` / `else` deciding which one to call. When both descriptions could plausibly answer the same question, the choice becomes sensitive to small wording differences in the prompt, and can vary run to run even at `temperature=0` for tool selection specifically (temperature affects text generation more predictably than tool-choice, which is worth confirming for yourself here).

The point of running the same ambiguous prompt 5 times isn't to find "the bug" — it's to see, directly, that an overlapping pair of descriptions produces a real, measurable split, not a single wrong answer you can just patch.

Write the two docstrings deliberately similar — e.g. `"Get the current weather for a city."` and `"Get the weather forecast for a city."` — close enough that a human skimming them might also be unsure. Tally the picks with a plain dictionary, like `{"get_weather": 0, "get_forecast": 0, "none": 0}`, and print a real number at the end, not just "it seemed to favor one."

A 5-run split like `{"get_weather": 3, "get_forecast": 2}` raises a harder question than "which one is right": is that split itself a problem, or is it just what a genuinely unclear prompt should produce? The prompt genuinely doesn't say whether "today" or "this week" was meant — a perfect model might legitimately split its answers too. The fix isn't always "make the split go away" — sometimes it's "recognize the prompt actually needs a follow-up question."

Hint 2 — The plan, and almost the whole thing

```
@tool
def get_weather(city: str) -> str: "Get the current weather for a city."

@tool
def get_forecast(city: str) -> str: "Get the weather forecast for a city."

function get_model_with_tools():
    return ChatOpenAI(...).bind_tools([get_weather, get_forecast])

function run_selection_trial(prompt, runs=5) -> dict:
    model_with_tools = get_model_with_tools()
    counts = {"get_weather": 0, "get_forecast": 0, "none": 0}
    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            picked = response.tool_calls[0]["name"]
        else:
            picked = "none"
        print(f"Run {run_number}: picked {picked}")
        counts[picked] = counts[picked] + 1
    return counts
```

Wire this into a full script with `if __name__ == "__main__":`, then compare against the Solution.

Full solution: Show me the solution